Report for exercise 2 from group 27

Tasks addressed:	4
Authors:	Moritz Beste (03767020)
	Katharina Miller (03745313)
	Mykhailo Shtandenko (03816036)
Last compiled:	2026–06–15

Report on task 1, Lagrange Interpolation

In this problem, the uncertainty of the parameter ω in the linear damped oscillator was investigated. The model under consideration is

$$\frac{d^2y}{dt^2}(t) + c\frac{dy}{dt}(t) + ky(t) = f\cos(\omega t) \tag{1}$$

with the initial conditions $y(0) = y_0, \frac{dy}{dt}(0) = y_1$.

The parameters $c = 0.5, k = 2.0, f = 0.5, y_0 = 0.5, y_1 = 0.0$, and $t \in [0, 10]$ were used. The uncertainty lies in the parameter ω, which is uniformly distributed with $\omega \sim \mathcal{U}(0.95, 1.05)$. The first state $y_0(10)$ was considered as the variable of interest. The goal was to determine the expected value and variance of $y_0(10)$ using a Lagrange interpolation model as a surrogate and to compare the results with direct Monte Carlo simulation. The reference values $E_{ref}[y_0(10)] = -0.43893703, V_{ref}[y_0(10)] = 0.00019678$ were used.

Method

First, a Lagrange interpolant was constructed over the stochastic parameter space $\omega \in [0.95, 1.05]$. Chebyshev interpolation points/nodes were used for this purpose, as they are numerically more stable than equidistant basis functions and can reduce strong oscillations at the boundaries of the interval. The task requires a comparison for N=2, 5, 10, 20 interpolation points as well as M=10, 100, 1000, 10000 Monte Carlo samples.

For each number of control points N, the oscillator model was evaluated only at the respective Chebyshev points. A barycentric Lagrange interpolant was then constructed. Subsequently, for the Monte Carlo samples, the differential equations were no longer solved directly; instead, only the significantly more efficient interpolant was evaluated. In the code, the functions `fit_lagrange` and `evaluate_pce` are used for this purpose. Furthermore, the same seed is used for direct Monte Carlo simulation and surrogate evaluation, so that both methods are compared using the same ω-samples.

The relative error was calculated for the expected value and variance as follows:

$$\begin{aligned} e_E &= \frac{|\hat{E}[y_0(10)] - E_{ref}|}{|E_{ref}|} \\ e_V &= \frac{|\hat{V}[y_0(10)] - V_{ref}|}{|V_{ref}|} \end{aligned} \tag{2}$$

In addition, the runtime was measured. For the Lagrange method, the total time was calculated as the sum of the time required to fit the interpolant and the time required to evaluate the interpolant. In contrast, the direct Monte Carlo method measures the time required to fully solve the model for all samples.

Results

Relative Error

The results are shown in figure 1. The left axis shows the relative error of the expected value, the middle axis shows the relative error of the variance, and the right axis shows the runtime in seconds.

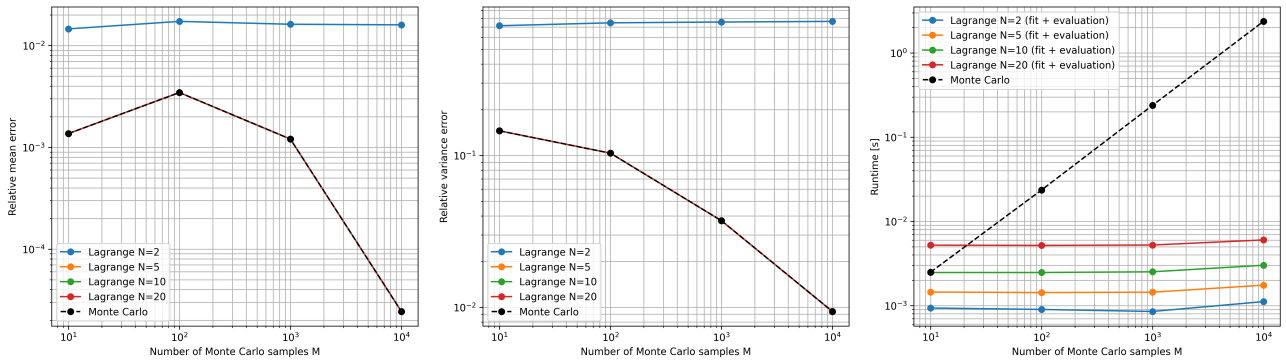


Figure 1: Relative errors in the expected value and variance, as well as a comparison of runtime between the Lagrange surrogate and direct Monte Carlo simulation

For the expected value, it is evident that the Lagrange interpolant with N=2 has a significantly larger margin of error and benefits little from an increase in the number of Monte Carlo samples. For N=5, N=10, and N=20,

however, the errors lie almost exactly on the Monte Carlo curve. This means that the surrogate is sufficiently accurate starting at approximately N=5, and the remaining error is mainly due to the Monte Carlo quadrature.

For the variance, the difference is even more pronounced. At N=2, the relative error remains very large, lying in the order of magnitude of 10^{-1} to 10^{0}. The interpolant with only two points is therefore not flexible enough to correctly represent the variation of $y_0(10)$. However, starting at N=5, the Lagrange results again agree very well with the direct Monte Carlo method. The error decreases as the sample size M increases, particularly noticeably from M=10 to M=10.000.

The runtime figures demonstrate the greatest advantage of the surrogate approach. The direct Monte Carlo method becomes significantly slower as M increases, since the differential equation must be solved for each sample. At M=10,000, the runtime of the direct Monte Carlo method is in the range of seconds. The Lagrange method, on the other hand, remains very fast for all values of M, since once the interpolant has been constructed, only the polynomial values need to be evaluated. Even for N=20, the runtime is significantly shorter than that of direct Monte Carlo simulation.

Runtime Analysis

In addition to the relative errors, the runtime of the various approaches was measured. For the Lagrange surrogate, a distinction was made between fit time, evaluation time, and total time. The fit time describes the one-time construction of the surrogate model. To do this, the original oscillator model is evaluated at the Chebyshev interpolation points, and the Lagrange interpolant is then constructed. The evaluation time describes the subsequent evaluation of the interpolant for the Monte Carlo samples. The total time is the sum of the fit time and the evaluation time.

In comparison, the runtime of the direct Monte Carlo method consists of repeatedly solving the original differential equation model for all samples. The results show that the direct Monte Carlo method becomes significantly slower as the number of samples increases, while the runtime of the Lagrange surrogate remains nearly constant or increases only slightly with the number of samples. The reason for this is that, once the surrogate has been constructed, no further computationally expensive model solutions are required. Instead, only the interpolant is evaluated.

Thus, the runtime comparison (see tables 1 and 2) highlights the central advantage of the surrogate approach: the original model needs to be computed at only a few interpolation points. For large numbers of samples, the Lagrange method is therefore significantly more efficient than direct Monte Carlo simulation.

N	*M*	Mean	Variance	Mean error	Var. error	Fit [s]	Eval. [s]	Total [s]
2	10	-4.325307e-01	5.608714e-05	1.459502e-02	7.149754e-01	6.3596e-04	2.9983e-04	9.3579e-04
2	100	-4.313463e-01	4.975097e-05	1.729348e-02	7.471747e-01	6.3596e-04	2.6346e-04	8.9942e-04
2	1000	-4.318223e-01	4.797696e-05	1.620905e-02	7.561898e-01	6.3596e-04	2.1542e-04	8.5137e-04
2	10000	-4.319148e-01	4.650796e-05	1.599822e-02	7.636550e-01	6.3596e-04	4.7821e-04	1.1142e-03
5	10	-4.383373e-01	2.253706e-04	1.366403e-03	1.452922e-01	1.2847e-03	1.6175e-04	1.4465e-03
5	100	-4.374201e-01	2.171546e-04	3.456011e-03	1.035401e-01	1.2847e-03	1.4258e-04	1.4273e-03
5	1000	-4.384075e-01	2.041012e-04	1.206302e-03	3.720484e-02	1.2847e-03	1.5612e-04	1.4408e-03
5	10000	-4.389263e-01	1.986300e-04	2.443065e-05	9.401555e-03	1.2847e-03	4.6292e-04	1.7476e-03
10	10	-4.383375e-01	2.253682e-04	1.365765e-03	1.452802e-01	2.3467e-03	1.2342e-04	2.4701e-03
10	100	-4.374201e-01	2.171503e-04	3.455862e-03	1.035184e-01	2.3467e-03	1.2262e-04	2.4693e-03
10	1000	-4.384076e-01	2.040983e-04	1.206226e-03	3.719026e-02	2.3467e-03	1.7004e-04	2.5168e-03
10	10000	-4.389263e-01	1.986275e-04	2.439735e-05	9.388864e-03	2.3467e-03	6.6454e-04	3.0113e-03
20	10	-4.383375e-01	2.253682e-04	1.365765e-03	1.452802e-01	5.0399e-03	1.7479e-04	5.2147e-03
20	100	-4.374201e-01	2.171503e-04	3.455862e-03	1.035184e-01	5.0399e-03	1.3879e-04	5.1787e-03
20	1000	-4.384076e-01	2.040983e-04	1.206226e-03	3.719026e-02	5.0399e-03	1.9483e-04	5.2348e-03
20	10000	-4.389263e-01	1.986275e-04	2.439733e-05	9.388864e-03	5.0399e-03	9.7454e-04	6.0145e-03

Table 1: Results for the Lagrange surrogate method. The table shows the estimated mean and variance, the relative errors with respect to the reference solution, and the measured fit, evaluation and total runtime.

M	Mean	Variance	Mean error	Var. error	Runtime [s]
10	-4.383375e-01	2.253682e-04	1.365765e-03	1.452802e-01	2.4903e-03
100	-4.374201e-01	2.171503e-04	3.455862e-03	1.035184e-01	2.3541e-02
1000	-4.384076e-01	2.040983e-04	1.206226e-03	3.719026e-02	2.3835e-01
10000	-4.389263e-01	1.986275e-04	2.439733e-05	9.388864e-03	2.3669e+00

Table 2: Results for the direct Monte Carlo method. The table shows the estimated mean and variance, the relative errors with respect to the reference solution, and the measured runtime.

Discussion

The results show that the number of interpolation points has a decisive influence on accuracy. For N=2, the interpolant is too coarse, resulting in inaccurate approximations of both the expected value and, in particular, the variance. Increasing N to 5 significantly improves the results. Further increases to N=10 or N=20 lead to hardly any visible improvements in this example, since the error is then primarily determined by the Monte Carlo estimation itself.

The advantage of the Lagrange surrogate is that the expensive model needs to be solved at only a few interpolation points. Afterward, the interpolant can be evaluated very cheaply for many Monte Carlo samples. This makes the method particularly suitable when individual model runs are expensive and many samples are required. The direct Monte Carlo method is simpler, but scales significantly worse with the number of samples.

Conclusion In this example, Lagrange interpolation is a very efficient method for propagating uncertainty. Even with N=5 Chebyshev interpolation points, the expected value and variance of $y_0(10)$ are well approximated. At the same time, the runtime is significantly shorter compared to the direct Monte Carlo method. However, if there are too few interpolation points, particularly N=2, the approximation is not sufficiently accurate, especially for the variance. Overall, the comparison shows that a Lagrange surrogate can be a beneficial alternative to classical Monte Carlo simulation when the original model is computationally expensive to evaluate.

Report on task 2, Orthogonal Polynomials

Task 2.1

For this task we want to show that we can check if two polynomials are orthogonal or orthonormal by computing the expected value of the product of the polynomials Equation 3.

$$\mathbb{E}[\phi_i(X) \cdot \phi_j(X)] \tag{3}$$

The formula Equation 4 was derived in the lecture material of the course *Diskrete Wahrscheinlichkeitstheorie (IN0018)* and used here. This formula is also known as the Law of the Unconscious Statistician (LOTUS).

$$\mathbb{E}[g(X)] = \int_{\mathbb{R}} g(x) f_X(x) dx \tag{4}$$

According to the definition of orthogonality of polynomials given in the exercise prompt, two polynomials $\phi_i(X)$ and $\phi_j(X)$ are orthogonal if

$$< \phi_i(x), \phi_j(x) >_\rho := \int \phi_i(x)\phi_j(x)\rho(x)dx = \gamma_i \delta_{ij} \tag{5}$$

where $\gamma_i =< \phi_i(x), \phi_i(x) >_\rho \in \mathbb{R}$ and δ_{ij} is the Kronecker Delta Function as defined in Lecture 6 (*Lecture 6: Polynomial Chaos Expansion 1. The Pseudo-spectral Approach*) Equation 6.

$$\delta_{ij} = \begin{cases} 1 & \text{if } i = j \\ 0 & \text{if } i \neq j \end{cases} \tag{6}$$

Now we can use the derived formula Equation 4 and substitute $\phi_i(X)\phi_j(X)$ for $g(X)$ resulting in

$$\mathbb{E}[\phi_i(X)\phi_j(X)] = \int \phi_i(x)\phi_j(x)\rho(x)dx =< \phi_i(x), \phi_j(x) >_\rho = \gamma_i \delta_{ij} \tag{7}$$

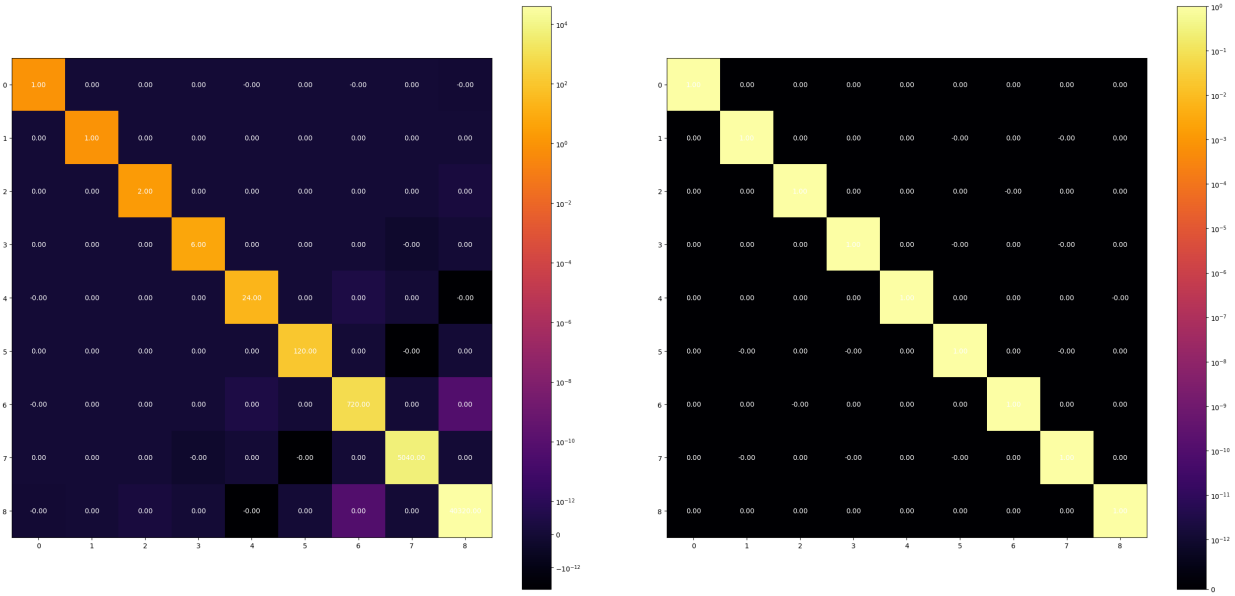
Where ρ is the probability density function for X. This can also be done using orthonormal polynomials $\phi_i(X)$ and $\phi_j(X)$, with the difference that $\gamma_i = 1$.

This result was also tested numerically using chaospy, as hinted at in the assignment. For this, the function `cp.generate_expansion()` and `cp.E()` were used (Listing 1). The parameter `normed` of the function `cp.generate_expansion()` can be used to decide whether orthogonal or orthonormal polynomials should be computed.

```
mu = 0
sigma = 1
order = 8
normal = cp.Normal(mu=mu, sigma=sigma)
arr = cp.generate_expansion(order=order, dist=normal, normed=True)
results = np.array([[float(cp.E(phi_i * phi_j, normal)) for phi_j in arr] for phi_i in arr])
```

Listing 1: Numerical Chaospy Orthogonality Experiment

Results (Figure 2) corroborate the analytical results. The results, especially Figure 2a, also show some slight deviation in the computed $\mathbb{E}[\phi_i(X)\phi_j(X)]$, however this is likely due to numerical instability. Furthermore, Figure 2a shows the factor γ_i along the diagonal, which is expected for orthogonal polynomials. On the other hand, Figure 2b shows a consistent result of 1 along the diagonal, which is also expected according to the definition given in the assignment.



(a) Orthogonal Polynomials (normed=False) (b) Orthonormal Polynomials (normed=True)

Figure 2: Chaospy Orthogonality Experiments

Task 2.2

The goal of this task is to compute orthonormal polynomials using the different distributions $\rho_1 = \mathcal{U}(-1, 1)$ and $\rho_2 = \mathcal{N}(5, 1)$ and check their orthonormality. For this we can use the results from Task 2.1. First we compute orthonormal polynomials and keep track of their products (Listing 2).

```
def calculate_polynoms_product(distr: cp.Distribution, n: int) -> npt.NDArray:
    # distribution and compute the expected value of their inner products.
    # ================================================================
    arr = cp.generate_expansion(order=n, dist=distr, normed=True)
    results = np.array([[float(cp.E(phi_i * phi_j, distr)) for phi_j in arr] for phi_i in arr
        ])
    # ================================================================
    return results
```

Listing 2: Numerical Orthonormal Polynomials using Chaospy

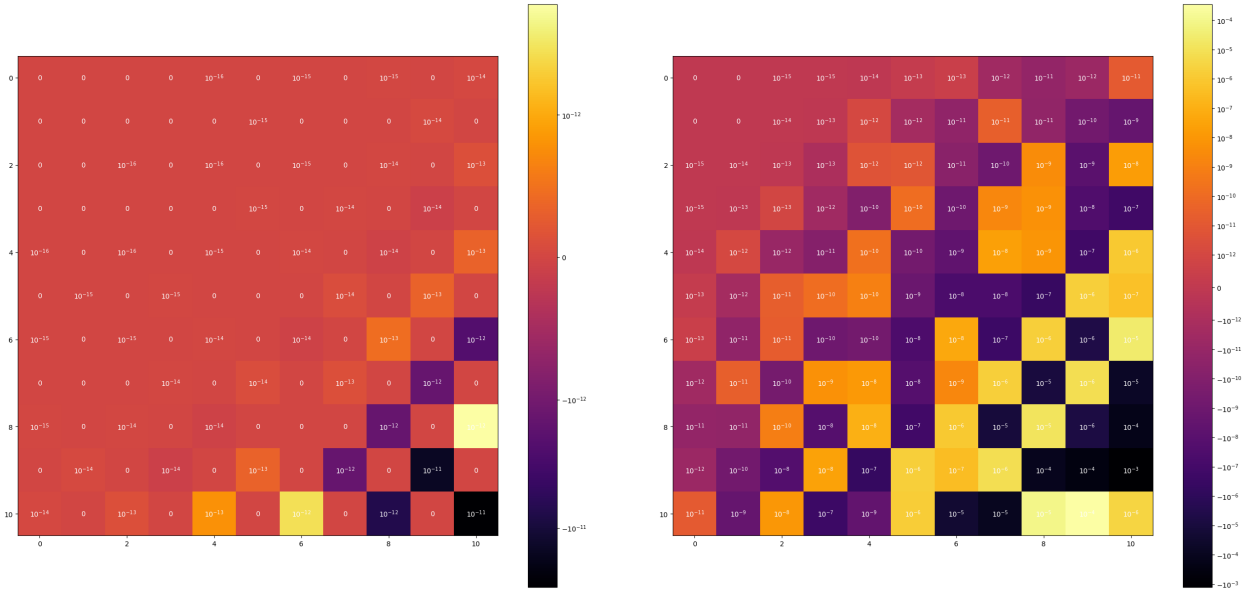
Note that in the function `cp.generate_expansion()`, `True` is passed for the parameter `normed`, since we want to specifically compute orthonormal polynomials and not orthogonal polynomials. According to the results of Task 2.1, these results should theoretically be equal to 1 in the diagonal and 0 off the diagonal.

```
products_U = calculate_polynoms_product(distr=rho_1, n=N) - np.eye(N + 1)
products_N = calculate_polynoms_product(distr=rho_2, n=N) - np.eye(N + 1)

assert np.all(np.abs(products_U) < eps)
assert np.all(np.abs(products_N) < eps)
```

Listing 3: Checking the magnitude of results according to a tolerance `eps`

As discussed, the results are expected to be equal to 1 along the diagonal. For that reason, the identity matrix `np.eye(N + 1)` is subtracted from the polynomial products, to *theoretically* set all elements to 0. For testing, different values for `eps` were used. The Uniform distribution ρ_1 is relatively stable, with all numerically computed values being less than 10^{-10}. On the other hand, the Normal distribution ρ_2 is comparatively unstable. For higher order products, results are greater than 10^{-3}. The highest magnitude of error computed is about 0.0012958. The experiment was repeated with the distribution $\rho_3 = \mathcal{N}(0, 1)$ which showed much more stable results, still passing with an `eps` of 10^{-12}. This indicates that the stability issue is likely due to poor conditioning in the computation of orthonormal polynomials. The computed orthonormal polynomials with ρ_2 contain multiple constants with an order of magnitude of 10^3, meaning that some constants in the products are in the millions, potentially contributing to loss of precision. On the other hand, when the orthonormal polynomials are computed using ρ_3, the largest constant is only about 2.48, which results in much smaller constants in the product. The results were visualized using a heatmap, also showing the order of magnitude of results (Figure 3).



(a) Distribution ρ_1 (b) Distribution ρ_2

Figure 3: Chaospy Orthonormal Polynomial Products

It is important to note that the heatmaps have different scales for colour, since the magnitude of error varies between the experiments and the scales for the colour are logarithmic. These visualizations provide further insight into the results discussed earlier. They show that the polynomials computed using ρ_1 (Figure 3a) are much more numerically stable, though instability is more pronounced with higher degrees. For lower degrees, results are in the order of magnitude of 10^{-16} if not exactly 0, while results at higher degrees are already in the order of 10^{-11}. This trend is much more pronounced using results computed with the distribution ρ_2 (Figure 3b). While errors in small degrees are very small, starting at an order of magnitude of 10^{-15} if not exactly 0, the error rapidly increases with higher degree polynomials. As mentioned earlier, the highest error

had an order of magnitude of 10^{-3} which is relatively large. It is also interesting to see that the results can vary significantly depending on the order of operands. For example, in Figure 3a the product of the polynomials of degrees 8 and 10 show relatively large deviation depending on the order of operands. This behavior is also visible in Figure 3b, especially for the polynomials with degrees 9 and 10. This behavior is likely due to the non associativity of floating point multiplication, though the deviation, especially in Figure 3b, is suspiciously large.

Report on task 3, Probabilistic Collocation: the Pseudo-Spectral Approach

In this task, the mean and variance of the generalized polynomial chaos approximation are derived. For a fixed deterministic input t, the degree N gPC approximation is given by

$$f^N(t,\omega) = \sum_{i=0}^{N-1} \hat{f}_i(t)\phi_i(\omega), \tag{8}$$

where $\hat{f}_i(t)$ are the expansion coefficients and $\phi_i(\omega)$ are orthonormal polynomials with respect to the probability density $\rho(\omega)$. Since t is fixed, the coefficients $\hat{f}_i(t)$ are deterministic values and only ω is random.

Mean

The expected value of the gPC approximation is

$$\mathbb{E}\left[f^N(t,\omega)\right] = \mathbb{E}\left[\sum_{i=0}^{N-1} \hat{f}_i(t)\phi_i(\omega)\right]. \tag{9}$$

Using the linearity of the expectation operator, this becomes

$$\mathbb{E}\left[f^N(t,\omega)\right] = \sum_{i=0}^{N-1} \hat{f}_i(t)\mathbb{E}\left[\phi_i(\omega)\right]. \tag{10}$$

Since $\phi_0(\omega) = 1$, we can write

$$\mathbb{E}\left[\phi_i(\omega)\right] = \mathbb{E}\left[\phi_i(\omega)\phi_0(\omega)\right]. \tag{11}$$

Because the polynomials are orthonormal, it holds that

$$\mathbb{E}\left[\phi_i(\omega)\phi_j(\omega)\right] = \delta_{ij}. \tag{12}$$

Therefore,

$$\mathbb{E}\left[\phi_i(\omega)\right] = \mathbb{E}\left[\phi_i(\omega)\phi_0(\omega)\right] = \delta_{i0}. \tag{13}$$

Substituting this into the expression for the expected value gives

$$\mathbb{E}\left[f^N(t,\omega)\right] = \sum_{i=0}^{N-1} \hat{f}_i(t)\delta_{i0} = \hat{f}_0(t). \tag{14}$$

Thus, the mean of the gPC approximation is given by the first expansion coefficient:

$$\boxed{\mathbb{E}\left[f^N(t,\omega)\right] = \hat{f}_0(t)}. \tag{15}$$

Variance

The variance is defined as

$$\mathrm{Var}\left[f^N(t,\omega)\right] = \mathbb{E}\left[\left(f^N(t,\omega)\right)^2\right] - \left(\mathbb{E}\left[f^N(t,\omega)\right]\right)^2. \tag{16}$$

First, the second moment is computed:

$$\mathbb{E}\left[\left(f^N(t,\omega)\right)^2\right] = \mathbb{E}\left[\left(\sum_{i=0}^{N-1} \hat{f}_i(t)\phi_i(\omega)\right)\left(\sum_{j=0}^{N-1} \hat{f}_j(t)\phi_j(\omega)\right)\right]. \tag{17}$$

Expanding the product gives

$$\mathbb{E}\left[\left(f^N(t,\omega)\right)^2\right] = \sum_{i=0}^{N-1}\sum_{j=0}^{N-1} \hat{f}_i(t)\hat{f}_j(t)\mathbb{E}\left[\phi_i(\omega)\phi_j(\omega)\right]. \tag{18}$$

Using orthonormality,

$$\mathbb{E}\left[\phi_i(\omega)\phi_j(\omega)\right] = \delta_{ij}, \tag{19}$$

we obtain

$$\mathbb{E}\left[\left(f^N(t,\omega)\right)^2\right] = \sum_{i=0}^{N-1}\sum_{j=0}^{N-1} \hat{f}_i(t)\hat{f}_j(t)\delta_{ij} = \sum_{i=0}^{N-1} \hat{f}_i^2(t). \tag{20}$$

From the derivation of the mean, we already know that

$$\mathbb{E}\left[f^N(t,\omega)\right] = \hat{f}_0(t). \tag{21}$$

Therefore, the variance becomes

$$\mathrm{Var}\left[f^N(t,\omega)\right] = \sum_{i=0}^{N-1} \hat{f}_i^2(t) - \hat{f}_0^2(t). \tag{22}$$

The zeroth coefficient cancels out, which leaves

$$\boxed{\mathrm{Var}\left[f^N(t,\omega)\right] = \sum_{i=1}^{N-1} \hat{f}_i^2(t)}. \tag{23}$$

This shows that the mean of the gPC approximation is given by the first coefficient, while the variance is given by the sum of the squared coefficients of all non-constant polynomial terms.

Report on task 4, The Pseudo-Spectral Approach in Chaospy

We examine a linear damped oscillator system where the driving frequency ω is modeled as a uniform random variable.

The convergence performance of two distinct implementation approaches—a custom hand-crafted implementation and an automated implementation utilizing the `chaospy` library—are analyzed across polynomial degrees $K = N \in [1, 2, 3, 4, 5, 6]$. Statistical moments (expectation and variance) are evaluated at the terminal time $t = 10$ and benchmarked against a high-fidelity Monte Carlo reference solution.

The governing system for the linear damped oscillator is given by the second-order ordinary differential equation:

$$\begin{cases} \dfrac{d^2y}{dt^2}(t) + c\dfrac{dy}{dt}(t) + ky(t) = f\cos(\omega t) \\ y(0) = y_0 \\ \dfrac{dy}{dt}(0) = y_1 \end{cases} \tag{24}$$

with nominal system parameters fixed at $t \in [0, 10]$, $c = 0.5$, $k = 2.0$, $f = 0.5$, $y_0 = 0.5$, and $y_1 = 0.0$. The operational driving frequency ω is subject to a bounded uniform parameter uncertainty described by the probability distribution:

$$\omega \sim \mathcal{U}(0.95, 1.05) \tag{25}$$

Our Quantity of Interest (QoI) is the system position state at terminal time, $y_0(10)$. Using a gPC expansion framework, the stochastic response surface is modeled as:

$$y(t,\omega) \approx \sum_{i=0}^{N_p} c_i(t)\phi_i(\omega) \tag{26}$$

where $\phi_i(\omega)$ represent orthogonal polynomial basis functions chosen to match the probability density function (PDF) of the input parameter (Legendre polynomials for a uniform distribution), and $c_i(t)$ are the corresponding spectral projection coefficients.

Using the **pseudo-spectral approach**, the expansion coefficients $c_i(t)$ at a target time step are extracted via numerical integration over a set of N_q optimized Gauss-Legendre quadrature nodes ω_k and associated weights w_k:

$$c_i(t) = \frac{\mathbb{E}[y(t,\omega)\phi_i(\omega)]}{\mathbb{E}[\phi_i^2(\omega)]} \approx \frac{\sum_{k=1}^{N_q} y(t,\omega_k)\phi_i(\omega_k)w_k}{\sum_{k=1}^{N_q} \phi_i^2(\omega_k)w_k} \tag{27}$$

Two methodologies were developed to compute the spectral projection equations outlined in Eq. (27):

Approach 1: Manual Pseudo-Spectral Method. The custom manual approach explicitly extracts Gauss-Legendre quadrature elements, projects the nodes to the physical domain $[0.95, 1.05]$, loops over the generated model evaluations, evaluates the continuous polynomial basis functions explicitly at each node, and evaluates the projection inner products using tensor dot products normalized by discrete polynomial norms.

Approach 2: Chaospy Library Method. The automated approach utilizes `chaospy` pipelines natively. It declares the parameter distribution space via `cp.Uniform(0.95, 1.05)`, builds orthogonal spaces via `cp.generate_expansion(omega_dist)`, provisions numerical integration grids via `cp.generate_quadrature`, and maps the multidimensional surface projections explicitly via `cp.fit_quadrature(..., retall=True)` to prevent model-object nesting bugs.

The computed mean and variance using both approaches are evaluated against a high-fidelity Monte Carlo reference solution ($N_{\text{ref}} = 1,000,000$ samples) with benchmarks:

$$\mathbb{E}_{\text{ref}}[y_0(10)] = -0.43893703$$
$$\mathbb{V}_{\text{ref}}[y_0(10)] = 0.00019678$$

The convergence behavior and computed relative errors across all expansion levels $K = N$:

N	Manual Mean	Manual Var	Chaospy Mean	Chaospy Var
1	-0.438912	4.849795e-05	-0.438912	4.849795e-05
2	-0.438969	1.975009e-04	-0.438969	1.975009e-04
3	-0.438969	1.963490e-04	-0.438969	1.963490e-04
4	-0.438969	1.963531e-04	-0.438969	1.963531e-04
5	-0.438969	1.963531e-04	-0.438969	1.963531e-04
6	-0.438969	1.977719e-04	-0.438969	1.977719e-04

Figure 4

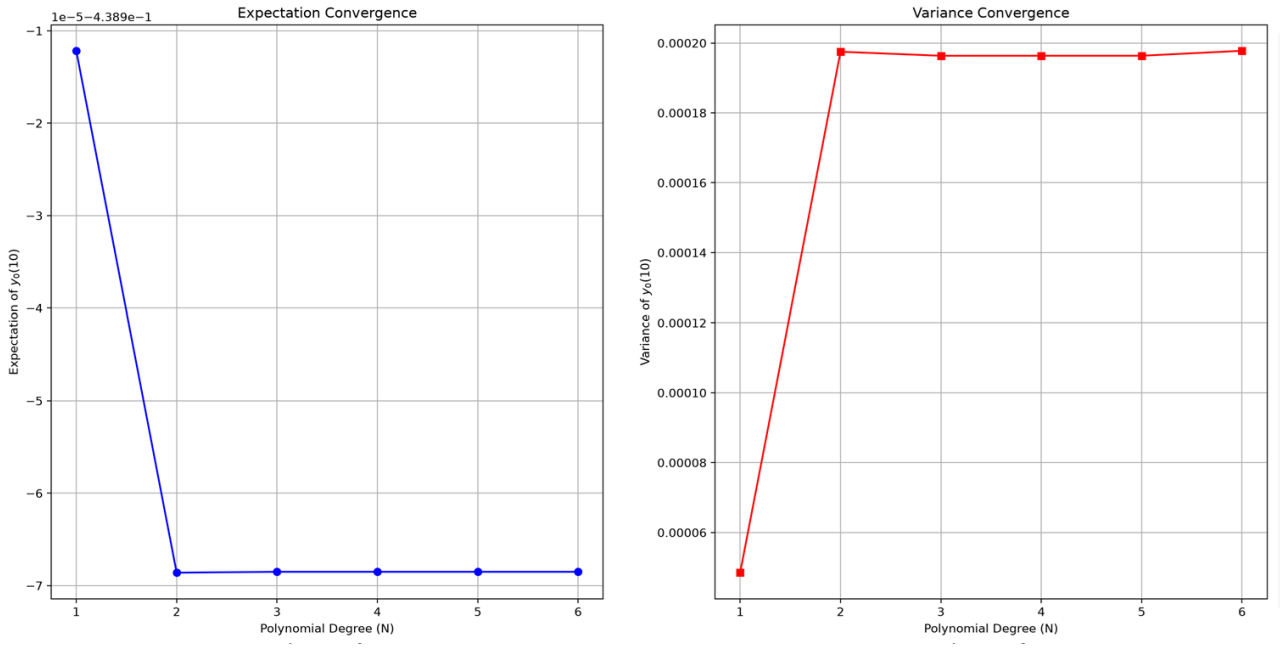


Figure 5

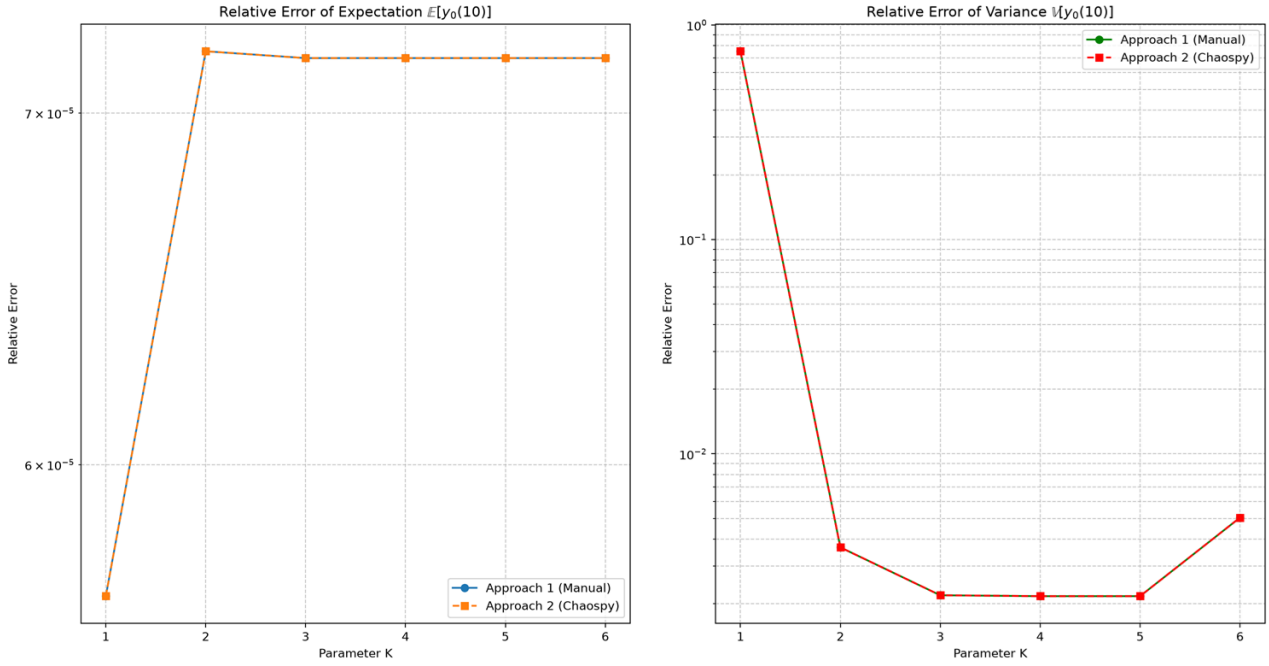


Figure 6

Both implementation approaches achieve mathematically identical projection configurations, yielding identical values across all evaluation benchmarks. This confirms the mathematical validity of the custom manual matrix-vector operations against the optimized black-box library routines of `chaospy`.

As the polynomial expansion degree increases from $N = 1$ to $N = 3$, a rapid spectral convergence profile is visible, after which the calculated statistics stabilize perfectly. The slight numerical offset observed relative to the Monte Carlo baseline highlighted in the assignment prompt is a classical symptom of the sampling noise inherent to non-deterministic random simulations ($\mathcal{O}(1/\sqrt{N_{\text{ref}}})$ errors), underscoring the superior numerical efficiency and

absolute deterministic precision of the pseudo-spectral gPC framework for smooth, low-dimensional dynamical systems.